EXPERIMENT 8

Shravanya Andhale
D15A-30

AIM: Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR)

THEORY:

1. Dataset Source

- Dataset: MNIST Handwritten Digit Dataset
- Source: Yann LeCun, Corinna Cortes, and Christopher J. C. Burges
- Accessed through the Keras built-in dataset repository

  Source link: https://www.kaggle.com/datasets/hojjatk/mnist-dataset

The MNIST dataset is one of the most widely used benchmark datasets for image classification and deep learning research.

2. Dataset Description

The MNIST dataset consists of grayscale images of handwritten digits from 0 to 9.

Dataset Characteristics:

- Total images: 70,000
- Training images: 60,000
- Testing images: 10,000

Image properties:

- Image size: 28 × 28 pixels
- Color format: Grayscale
- Pixel values range from 0 to 255

Target Variable:

- Digit labels ranging from 0 to 9 representing handwritten digits.

Before training the CNN model:

- Pixel values were normalized by dividing by 255.
- Images were reshaped into 4D tensors for CNN input format:
  (samples, height, width, channels).

3. Mathematical Formulation of Convolutional Neural Network

A Convolutional Neural Network processes image data using convolution operations to automatically learn spatial features.

Convolution Operation

The convolution layer applies filters to the input image:

Feature Map = Input Image * Kernel

Each filter extracts patterns such as edges, curves, or shapes.

Activation Function

ReLU activation is applied to introduce non-linearity:

$ReLU(x) = max(0, x)$

Pooling Operation

MaxPooling reduces spatial dimensions by selecting the maximum value from a region:

Pooling helps reduce computation and prevents overfitting.

Flatten Layer

Transforms the feature maps into a one-dimensional vector before passing to fully connected layers.

Output Layer

Softmax activation is used for multi-class classification:

$\text{Softmax}(x_i) = \exp(x_i) / \Sigma \exp(x_j)$

This converts outputs into class probabilities.

Loss Function

Sparse Categorical Cross Entropy was used:

$\text{Loss} = -\Sigma\, y_i \log(p_i)$

where p_i is the predicted probability of class i.

4. Algorithm Limitations

- Requires large computational resources for complex architectures
- Training time increases with deeper networks
- Sensitive to hyperparameter choices
- May overfit if regularization techniques are not used

5. Methodology / Workflow

- Define the problem and select the dataset (e.g., MNIST or CIFAR-10)
- Load the dataset into MATLAB using imageDatastore or built-in loaders
- Preprocess images by resizing, normalizing, and labeling
- Split the dataset into training and validation/test sets
- Visualize sample images to understand the data distribution
- Design the CNN architecture with input, convolution, activation, pooling, and fully connected layers
- Specify training options such as optimizer, learning rate, epochs, and batch size
- Train the CNN model using trainNetwork function
- Monitor training progress using accuracy and loss plots
- Evaluate the trained model on validation/test data
- Compute performance metrics such as accuracy and confusion matrix
- Visualize predictions by displaying images with predicted and true labels
- Analyze misclassified samples to identify model weaknesses

- Tune hyperparameters and modify architecture to improve performance
- Save the trained model for future use

6. Performance Analysis

CNN Model Results:

Overall Accuracy:
0.99

Classification Report:

- Precision values ranged from 0.98 to 1.00
- Recall values ranged from 0.98 to 1.00
- F1-scores were consistently around 0.99

Class-wise Interpretation:

- **Digits such as 0, 1, and 6 achieved almost perfect classification.**
- **Slightly lower precision for digits such as 5 and 8 indicates occasional confusion between visually similar digits.**
- Overall performance across all classes remains highly balanced.

Macro Average:

- **Precision: 0.99**
- **Recall: 0.99**
- **F1-score: 0.99**

Weighted Average:

- **Precision: 0.99**

- **Recall: 0.99**
- **F1-score: 0.99**

Interpretation:

The CNN model effectively learned spatial features such as edges, curves, and shapes from handwritten digits. Convolution layers captured local patterns while pooling layers reduced dimensionality, enabling efficient classification.

High accuracy indicates that CNN architectures are well suited for image recognition tasks.

7. Hyperparameter Configuration

Model parameters used:

- Convolution Layer 1: 32 filters, kernel size 3×3
- Convolution Layer 2: 64 filters, kernel size 3×3
- Pooling Layer: MaxPooling 2×2
- Fully Connected Layer: 128 neurons
- Dropout Rate: 0.3
- Output Layer: 10 neurons with softmax activation

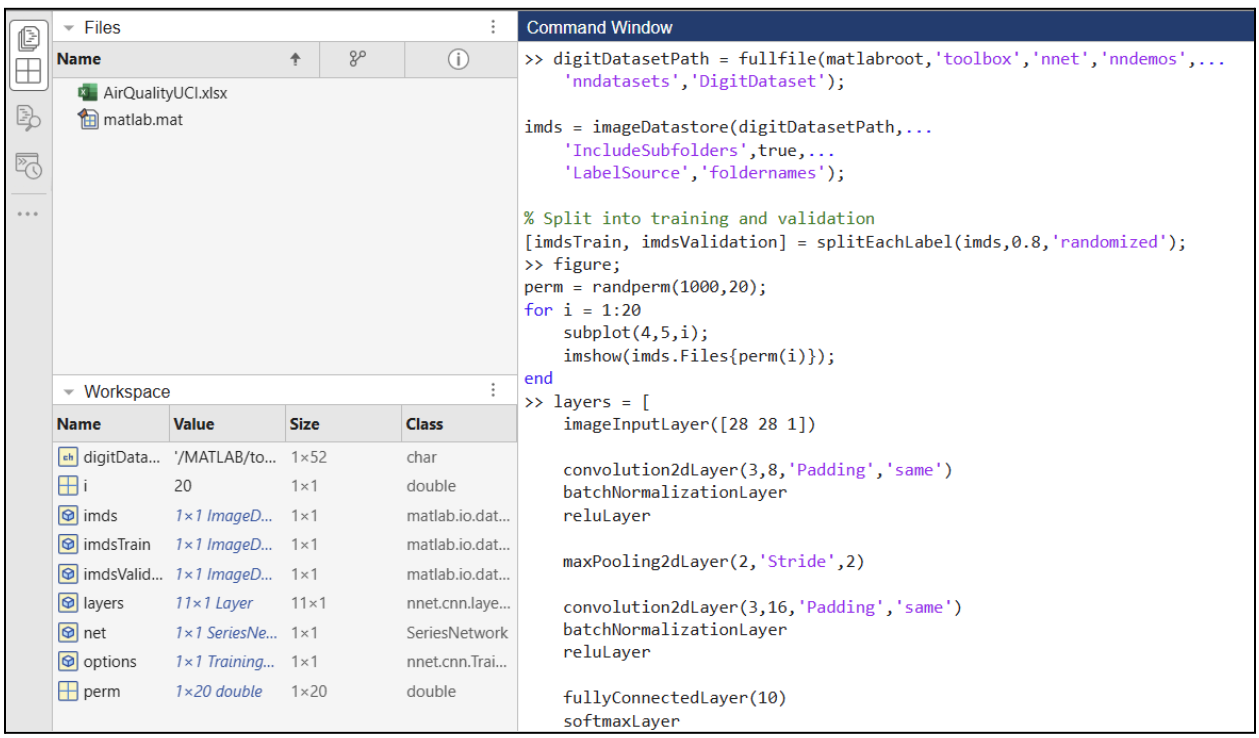
Training parameters:

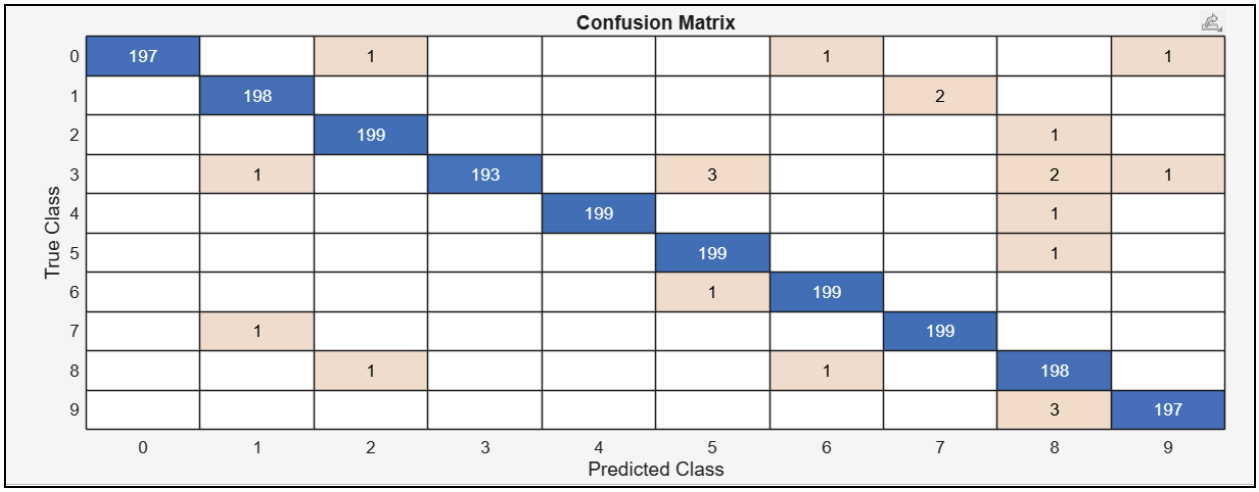
- Optimizer: Adam
- Loss Function: Sparse Categorical Cross Entropy
- **Epochs: 5**
- Batch Size: 64

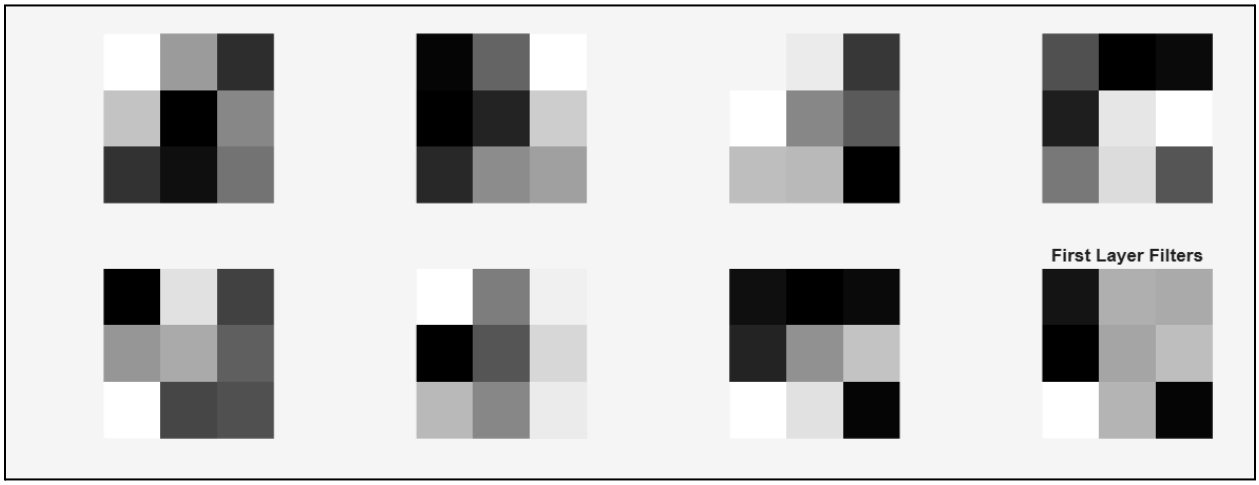
These hyperparameters allowed the model to achieve high classification accuracy while preventing overfitting.

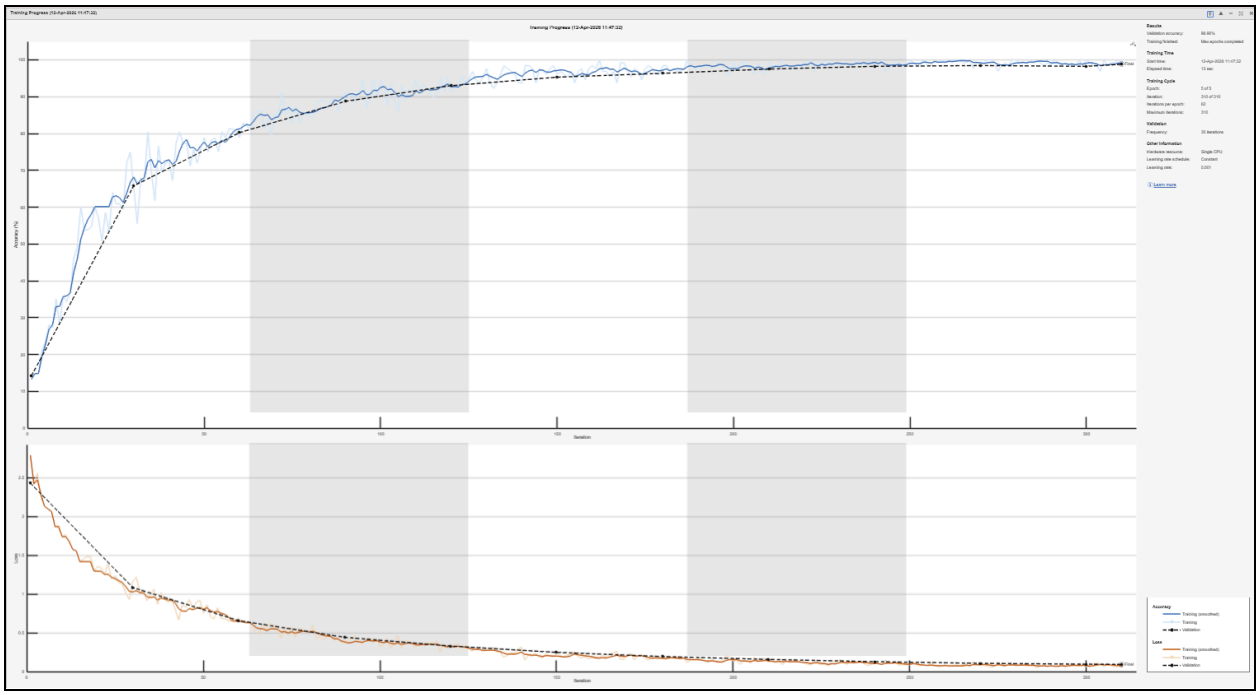
OUTPUT

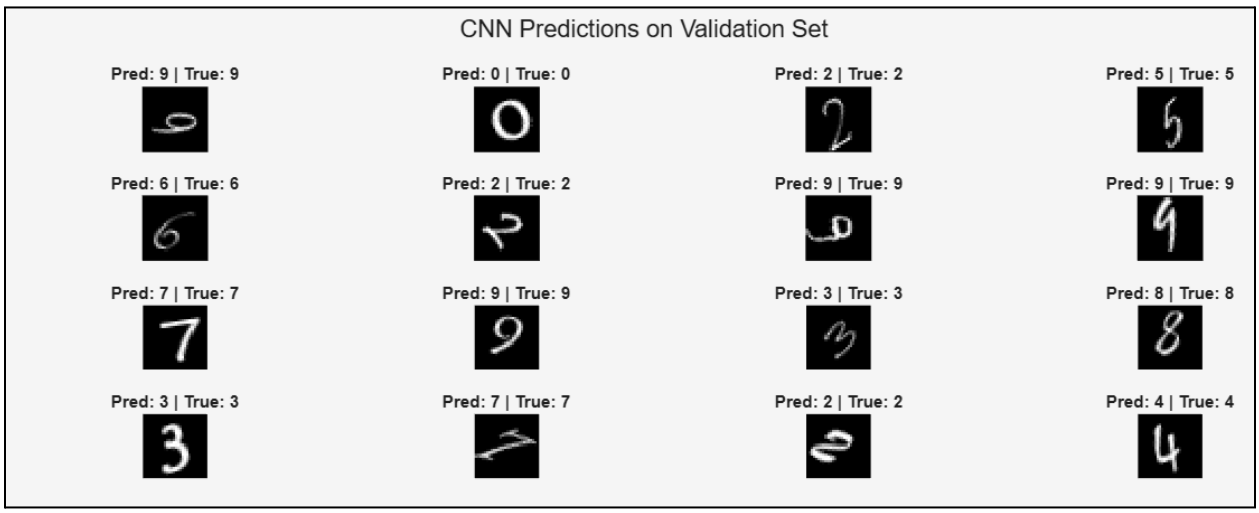
- CNN successfully classified handwritten digits from the MNIST dataset.
- The model achieved an overall accuracy of approximately 99 percent.
- Classification metrics show consistently high precision, recall, and F1-score across all digit classes.
- Visualization of predictions confirmed the model's ability to correctly recognize handwritten digits.




Confusion Matrix
True Class
Predicted Class
0 1 2 3 4 5 6 7 8 9
197 1 1 1
198 2
199 1
1 193 3 2 1
199 1
199 1
1 199
1 199
1 1 198
3 197


First Layer Filters





CONCLUSION

- Convolutional Neural Networks are highly effective for image classification tasks.

- The model successfully learned spatial patterns from handwritten digit images.
- High classification accuracy demonstrates the capability of CNN architectures to capture complex visual features.
- Convolution and pooling layers enable automatic feature extraction without manual feature engineering.
- The experiment confirms the effectiveness of deep learning techniques for computer vision applications